

RemMux: Sistem Client–Server pentru Execuția Concurentă a Comenzilor Remote

Pleşca George-Claudiu

Facultatea de Informatica, Universitatea Alexandru Ioan Cuza din Iași
`george-claudiu.plesca@uaic.info.ro`

Abstract. RemMux este o aplicatie client-server concurenta, ce permite executarea de comenzi la distanta. Clientul are posibilitatea de a deschide ferestre multiple, prin intermediul carora va comunica cu server-ul.

1 Introducere

În cele ce urmează urmează să descriu proiectul "Remmux". În ce constă RemMux mai exact? Acesta este o aplicație client/server, prin intermediul căreia mulți clienți pot executa comenzi de secure shell la distanță. Fiecare client are la dispoziție o interfață grafică în intermediul terminalului care îi permite afișarea de ferestre multiple și executarea de comenzi din acesta, asemănător cu Tmux.

2 Tehnologii utilizate

Pentru implementarea proiectului am decis să folosesc un server concurent TCP/IP care creează un thread pentru fiecare comandă pe care clientul o trimite serverului, permitând execuția simultană a comenzilor de la mai mulți clienți.

TCP-ul permite transmiterea completă a informațiilor, în ordine, fără pierderi pe parcurs asigurând astfel ca server-ul primește fiecare comandă fără pierderi pe parcurs și ca fiecare client primește informația corectă și completă.

Clientul are la dispoziție o interfață grafică în cadrul terminalului, care permite afișarea de ferestre multiple prin intermediul cărora se pot trimite sau pot fi recepționate mesaje de la server. Manipularea interfeței se poate realiza și dacă server-ul este offline, clientul având posibilitatea de a crea până la 6 ferestre simultane, de a redimensiona ecranul și de a da scroll la textul scris anterior.

3 Structura aplicației

Aplicația este structurată pe o arhitectură client-server TCP cu următoarele concepte de modelare:

Modularizare si separarea responsabilitatilor

Fiecare etapa din fluxul de procesare al comenzii este gestionat o functie distincta, care permite o organizare cat mai buna a responsabilitatilor:

- preluarea comenzilor de la client (raspunde)
- determinarea tipului de comenzi (simple sau multiple) si pregatirea lor pentru executie (exec_cmd)
- executarea comenzilor simple (exec_sg_cmd, exec_generic_cmd)
- executarea comenzilor multiple (executa_mult_cmd)
- executarea operatiilor in cadrul comenzilor multiple (ORwell, pipeline, AND_oftheworld, DotCom, Redirect_SingleOutput)
- comunicarea cu clientul asupra rezultatului final (returneaza)
- pregatirea si partajarea rezultarelor catre client (return2cl)

Concurenta

Aceasta se realizeaza cu ajutorul unui model thread-per-client. Serverul creeaza un thread pentru fiecare client, prin care se ruleaza procesarea si executia comenzilor clientului, ca la final sa fie detasat automat de la server, permitand lucrul simultan a mai multor clienti cu multiple ferestre fara blocaje.

Gestionare comenzilor

Programul dispune de o clasa personalizata "Commandments", care asigura o gestionare flexibila a comenzilor primite de la client. Acesta asigura:

- Determinarea operatorilor (return_operation)
- Determinarea comenzii (return_cmd)
- Determinarea operatorilor care afecteaza redirectionarea erorilor (2>), output-ului (>) sau inputului (<)
- Determinarea comenzilor cu functionabilitati speciale in contextul serverului (detect_custom_cmd)
- Convertirea comenzilor pentru a le pregati de executie (char_convert)

Executarea comenzilor

Executarea comenzilor este realizata printr-un mecanism bazat pe procese UNIX

- Fiecare comanda este executata intr-un proces separate (fork)
- Se utilizeaza **execv** pentru executarea comenzilor
- Redirectionarea (erorilor) se realizeaza cu **dup2** in functie de context (redirectionare catre client, catre un fisier output, citirea de la client, citirea de la un fisier input).
- Sincronizarea se realizeaza cu **waitpid** in mod nonblocant, cu suport timeout pentru comenzi care nu se pot executa in timp

Interfata cu utilizatorul

Programul dispune de o clasa personalizata "my_windows" bazata pe biblioteca ncurses care asigura o gestionare cat mai complexa a interfetei din terminal. Acesta permite, printre altele:

- Creerea de ferestre noi (Create_Window)
- Adaugarea de noi ferestre (Add_Window)
- Un istoric asociat fiecarei ferestre (CreateWindowHistory) care asigura faptul ca nu vom pierde informatiile din ferestre cand vor fi redimensionate
- Restaurarea ferestrei (Restore_Window)
- Schimbarea ferestre curente (Change_CurrWindow)
- Scroll (scrollup / scrolldown)
- Redimensionarea ferestrei (resize_win)

Intr-un loop while, utilizatorul trimite comenzi catre ferestre prin intermediul tastaturii:

- **Enter** - asigura conectarea la server si daca e realizata cu success, utilizatorul poate scrie comenzi catre acesta.
- **+/=** - adauga ferestre noi
- **tab** - schimba fereastra
- **q** - iesirea din program
- **MouseScrollUp/Arrow Up** - Scroll up
- **MouseScrollDown/Arrow Down** - Scroll down

```

georgepa2@georgepa2-Thin-A15-B7VF: ~/Documents/Proiect-RemMux_C++

Press <<Enter>> to start typing, '+' to create a new window <<tab>> to switch between windows or 'q' to quit!
[Server] V-ati conectat cu succes!
[client]Introduceti o comanda: pwd
[Server] /home/georgepa2/Documents/Proiect-RemMux_C++
[Client] Do you want continue? y/n

[client]Introduceti o comanda: ls | head -n5
[Server] Client_logs
clientRemMux
clientRemMux.cpp
erorr.txt
error.txt
[Client] Do you want continue? y/n
[client]Introduceti o comanda: touch aa.txt
[Server] Comanda a fost executata cu succes!
[Client] Do you want continue? y/n

[client]Introduceti o comanda: ls | head -n1
&& echo a mers || echo nu a mers
[Server]aa.txt
a mers
[Client] Do you want continue? y/n

```

Fig. 1. O captura de ecran a interfeței cu ferestre multiple de pe care clientul trimite comenzi către server

```

georgepa2@georgepa2-Thin-A15-B7VF: ~/Documents/Pr...

Press <<Enter>> to start typing, '+' to create a new window <<tab>> to switch between windows or 'q' to quit!
[Server] V-ati conectat cu succes!
[client]Introduceti o comanda: pwd
[Server] /home/georgepa2/Documents/Proiect-RemMux_C++
[Client] Do you want continue? y/n

clientRemMux.cpp
erorr.txt
error.txt
[Client] Do you want continue? y/n

head -n1 && echo a mers || echo nu a mers
[Server] aa.txt
a mers

```

Fig. 2. O captura de ecran a aceleasi interfețe, în urma redimensionării ecranului. Se poate observa și faptul că conținutul ecranului a fost scrollat în sus de către client

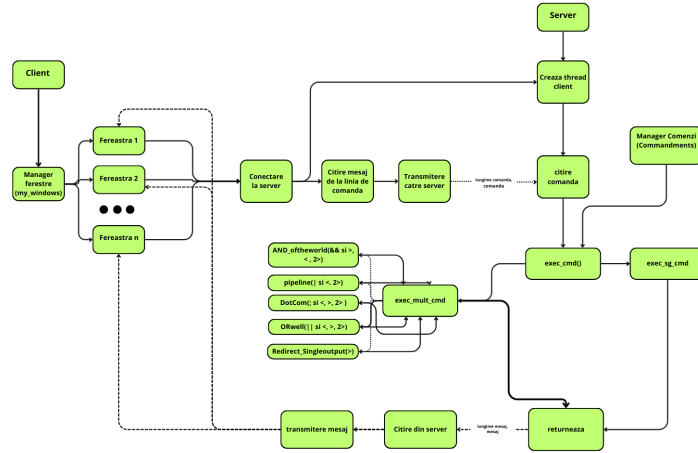


Fig. 3. Diagrama RemMux

4 Protocolul de comunicare

Trimiterea comenzii de catre client catre server

1. Clientul citeste o comanda scrisa de catre utilizator de la tastatura din orice fereastră aleasa de catre acesta.
2. Clientul calculeaza lungimea mesajului.
3. Clientul transmite catre server un numar întreg de 4 octeti, care reprezinta lungimea mesajului si un sir de caractere care reprezinta comanda in sine.

Receptionarea comenzii de catre server

1. Server-ul citeste de la client cei 4 octeti corespunzatori lungimii comenzii.
2. Server-ul alocă memoria necesara pentru receptionarea comenzii.
3. Server-ul citeste sirul de caractere (comanda) trimis de catre client.

Procesarea comenzilor in server

1. Server-ul verifica daca comanda este simpla sau formata din comenzi multiple
2. In cazul in care comanda este simpla, acesta il executa direct
3. In cazul comenzilor multiple, acestea sunt executate intr-un loop pana la epuizarea comenzilor
4. Server-ul calculeaza lungimea output-ului comenzii sau al unui mesaj generic care anunta executarea comenzii.
5. Server-ul transmite clientului un număr întreg de 4 octeți (lungimea răspunsului) și șirul de caractere – mesajul raspuns.

Recepționarea răspunsului de către client

1. Clientul citește de la server cei 4 octeți ai lungimii răspunsului.
2. Clientul alocă spațiul necesar pentru mesaj.
3. Clientul citește mesajul și îl afișează în fereastra corespunzătoare.
4. Clientul revine la starea de așteptare a unei noi comenzi (conectare, schimbat fereastra, adăugare fereastră sau quit).

Gestionarea situațiilor excepționale

Dacă clientul se deconectează în timpul transmiterii mesajului:

1. Server-ul detectează eroarea.
2. Thread-ul alocat clientului deconectat se închide, fără a perturba activitatea celorlalți clienți.

Dacă server-ul cade în timp ce clientul tastează:

1. Clientul blochează recepționarea semnalelor de la server.
2. Clientul detectează eroarea.
3. Clientul afișează un mesaj local de eroare și revine la starea de așteptare de comenzi noi.

5 Scenarii de utilizare

Scenarii de succes:

1. **Clientul trimite o comanda valida catre server, de pe o fereastră oarecare:** clientul trimite, spre exemplu, comanda „ls | wc -l” din fereastra 3 către server; acesta creează un thread nou pentru fereastra numărul 3 a clientului, executa comenzile și va transmite output-ul către client, acesta fiind afișat în fereastra corespunzătoare
2. **Utilizatorul adaugă o fereastră nouă, după ce a executat mai multe comenzi anterior într-o altă fereastră:** Clientul verifică dacă s-a atins numărul maxim de ferestre; dacă da — nu face nimic, altfel schimbă dimensiunile ferestrei anterioare și adaugă pe cea nouă, asigurând că vechea fereastră își păstrează conținutul anterior.
3. **Mai mulți clienți trimit comenzi către server:** Fie un ClientA care trimite „pwd”, și un ClientB care trimite „whoami”. Server-ul creează câte un thread separat pentru fiecare; fiecare thread creează un fișier temporar corespunzător pid-ului si id-ul sau. Fiecare client va recepționa mesajul corespunzător comenzii tastate, fără să fie nevoit să aștepte pe celălalt și fără să existe interferențe între ele.

Scenarii de eșec:

1. **Clientul se conectează la server, dar acesta este offline:** Clientul încearcă să se conecteze la server; serverul fiind offline, clientul primește un mesaj local de eroare și se întoarce în fereastră după 5 secunde, de unde are posibilitatea să aștepte serverul să revină, să creeze ferestre noi sau să navigheze printre ferestre.
2. **Server-ul cade în timp ce clientul tastează o comandă:** Clientul s-a conectat cu succes la server, însă acesta pică chiar în timp ce clientul tastează o comandă. Clientul blochează semnalele de închidere pe care server-ul i le-ar putea trimite, primește un mesaj local de eroare, revine la fereastră, iar semnalele sunt deblocate.
3. **Un client scrie o comanda care nu se va termina niciodata:** Serverul activează un timer pentru fiecare comanda. Dacă trec mai mult de 10 secunde, server-ul transmite un mesaj corespunzător către client, fără a afecta ceilalți clienți conectați la server și asigurând că utilizatorul poate continua să folosească interfața-client

6 Concluzii

În mare parte, îmi pare rău că nu am putut lucra mai mult la proiect. Sunt aspecte care ar mai fi beneficiat de un polish mai mare, sau mecanici pe care am dorit să le implementez, însă în mare proiectul în sine a fost o experiență bună, chiar dacă a fost și puțin stresantă. Am învățat cât de mult contează să te organizezi și să-ți faci planuri din timp, pentru a facilita procesul muncii, sunt multe momente în care a trebuit să rescriu bucăți de cod întregi sau să scriu funcții mari care s-au dovedit a fi inutile din cauza lipsei de planificare și organizare, nu a fost foarte distractiv. Timpul a fost cel mai presant factor al proiectului, în rest, orice problemă are o soluție, chiar dacă uneori sunt mai radicale.

Dacă ar fi să îmbunătățesc proiectul în viitor, în primul rând aș acorda o atenție mai mare asupra fixării de bug-uri și a implementării de features care lipsesc, ca mai apoi să adaug funcționalitate pentru anumite comenzi care aveau probleme și de a continua în a îmbunătăți interfața cu ferestre ncurses, adăugând mai multe opțiuni și comenzi.

References

1. Template server concurrent și client — Universitatea „Alexandru Ioan Cuza” Iași, Curs Rețele de Calculatoare. <https://edu.info.uaic.ro/computer-networks/files/NetEx/S12/ServerConcThread/servTcpConcTh2.c> <https://edu.info.uaic.ro/computer-networks/files/NetEx/S12/ServerConcThread/cliTcpNr.c>
2. Pradeep Padala — *NCURSES Programming HOWTO*. <https://tldp.org/HOWTO/NCURSES-Programming-HOWTO/index.html>
3. invisible-island.net/ncurses/man/ <https://invisible-island.net/ncurses/man/>

4. Casual Coder — playlist tutorial ncurses. https://www.youtube.com/playlist?list=PL2U2TQ__OrQ8jTf0_noNKtHMuYlyxQ14v
5. Geeks for Geeks — *typedef in C*. <https://www.geeksforgeeks.org/c/typedef-in-c/>
6. CPlusPlus.com — referinţe pentru `std::string`, `std::vector` si `std::vector`. <https://cplusplus.com/reference/string/string/> <https://cplusplus.com/reference/vector/vector/?kw=vector> <https://cplusplus.com/reference/map/map/>
7. Canva — *template diagrama* <https://www.canva.com/graphs/diagrams/>
8. Stackoverflow — *Redirect perror output to fprintf(stderr, "")* <https://stackoverflow.com/questions/5483120/redirect-perror-output-to-fprintfstderr>
9. Stackoverflow — *Ncurses and Resizing window* <https://stackoverflow.com/questions/29717192/ncurses-and-resizing-window>
10. Stackoverflow — *Ncurses and Resizing window* <https://stackoverflow.com/questions/29717192/ncurses-and-resizing-window>
11. Programare: lucrul cu mai multe procese secvenţiale (II): apelurile din familia `exec` - Universitatea „Alexandru Ioan Cuza” Iaşi, *Curs Sisteme de Operare*. https://edu.info.uaic.ro/sisteme-de-operare/S0/support-lessons/C/suport_lab11.html